Yet Another Audio analyzeR (YAAR): A Real-Time FFT-Based Audio Measurement Tool for Distortion, Noise, and Power Analysis

George Biro
Independent Developer
GPLv3-or-later
Email: george.biro@gmail.com

***Abstract*—This paper presents Yet Another Audio analyzeR (YAAR), a lightweight command-line and interactive plotting tool for real-time audio analysis, together with a separate signal generator utility for deterministic test signal synthesis. The analyzer combines time-domain monitoring, FFT-based spectral estimation, rolling magnitude averaging, automatic tone detection, and measurement of total harmonic distortion (THD), total harmonic distortion plus noise (THD+N), intermodulation distortion (IMD), signal-to-noise ratio (SNR), signal-to-noise-and-distortion ratio (SINAD), effective number of bits (ENOB), and electrical output metrics such as V_{RMS} and P_{RMS}.**

YAAR supports both live acquisition from an audio back-end and an internal simulation mode, while the accompanying generator produces phase-continuous single-tone and two-tone excitation signals for controlled measurements. The two programs can be used independently or combined to form a complete measurement chain, enabling real-time analysis, repeatable simulation, and end-to-end validation of audio systems.

The implementation was validated on a MacBook Air M4 using a Cosmos ADC (E1DA), a UNI-T UTG932E signal generator, and a FiiO BR3 Bluetooth device used as a DAC.

This document serves as a professional user and technical manual, describing the operating principles, command-line interfaces, measurement workflow, configuration parameters, and practical usage recommendations.

***Index Terms*—audio measurement, FFT, THD, THD+N, IMD, SNR, SINAD, ENOB, spectrum analysis, Python**

I. Introduction

Audio test workflows often require a compact toolset that can both generate controlled excitation signals and analyze responses in real time, without the overhead of a full laboratory software suite. YAAR addresses the analysis side of this requirement with a lightweight Python implementation centered on FFT-based processing and a clear operational model.

In addition to the analyzer, the project includes a separate signal generator utility, enabling deterministic single-tone and two-tone stimulus generation. The two programs can be used independently or combined to form a complete measurement chain, supporting both purely simulated scenarios and end-to-end hardware validation.

The analyzer supports either a physical input stream or an internal simulation path, making it suitable for both bench measurements and algorithm development, while the external generator enables validation of real signal paths including DACs, amplifiers, and ADCs.

The software is designed around the following goals:

1) real-time visualization of the waveform and spectrum,
2) robust spectral metric extraction under single-tone and two-tone conditions,
3) deterministic signal generation for repeatable test scenarios,
4) simple command-line configuration,
5) CSV and plot export for reproducible reporting.

II. System Overview

At runtime, YAAR repeatedly acquires or synthesizes a measurement block, applies a selected window function, computes a real FFT, updates a rolling spectrum average, detects one or two dominant tones, builds masks for the fundamental and distortion components, and derives scalar metrics for display and export.

III. Software Architecture

The main program logic is organized around a continuous measurement loop. Several helper functions and lightweight classes encapsulate reusable behavior:

- **`simulate_signal`** generates one-tone or two-tone synthetic signals with optional additive noise and distortion.
- **`RollingFFTAverage`** smooths frame-to-frame FFT magnitude variation by averaging power spectra over a sliding window.
- **`WRef`** constructs a spectral kernel corresponding to a windowed sinusoid and shifts it to expected tone bins.
- **Peak detection and mask builders** determine whether the measurement is best interpreted as THD or IMD.
- **Metric functions** convert partitioned spectral powers into engineering quantities such as THD, THD+N, SNR, and ENOB.

IV. Measurement Theory

A. FFT Magnitude Estimation

Given a measurement vector $x[n]$ of length N and a window $w[n]$, YAAR computes the real FFT of $x[n]w[n]$. The resulting magnitude spectrum is corrected for window coherent gain and equivalent noise bandwidth (ENBW):

$$|X[k]| = \frac{|\mathrm{RFFT}\{x[n]w[n]\}|}{N\,G_c\,\sqrt{\mathrm{ENBW}}} \tag{1}$$

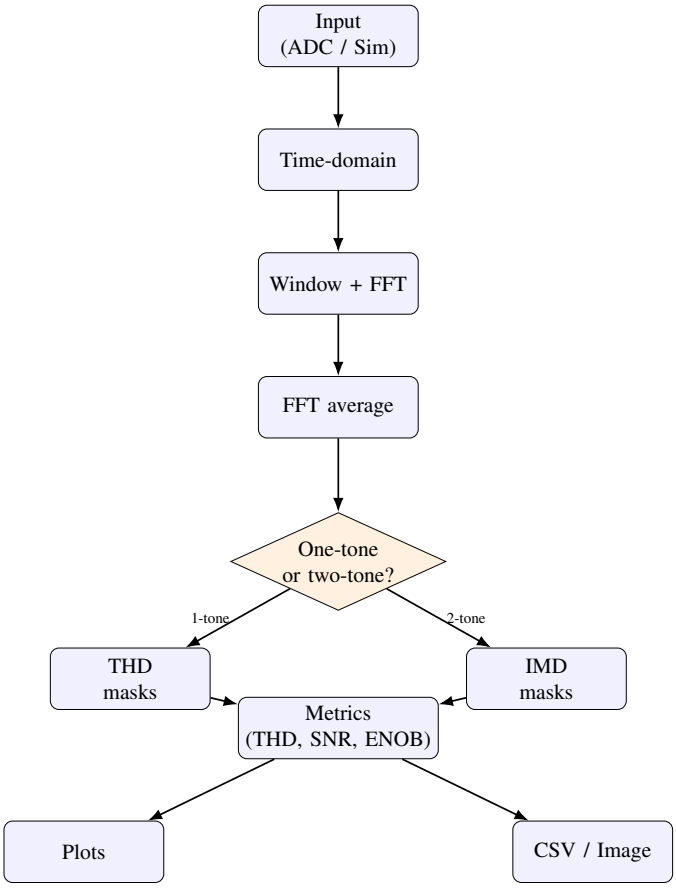

Fig. 1. High-level YAAR processing flow.

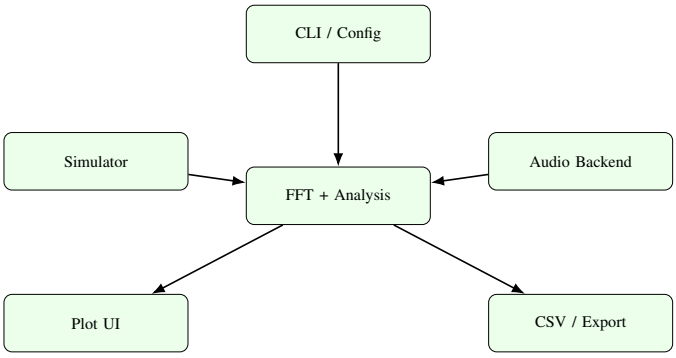

Fig. 2. Functional organization of YAAR.

where

$$G_c = \frac{1}{N}\sum_{n=0}^{N-1} w[n] \tag{2}$$

and

$$\mathrm{ENBW} = N\frac{\sum_n w[n]^2}{\left(\sum_n w[n]\right)^2}. \tag{3}$$

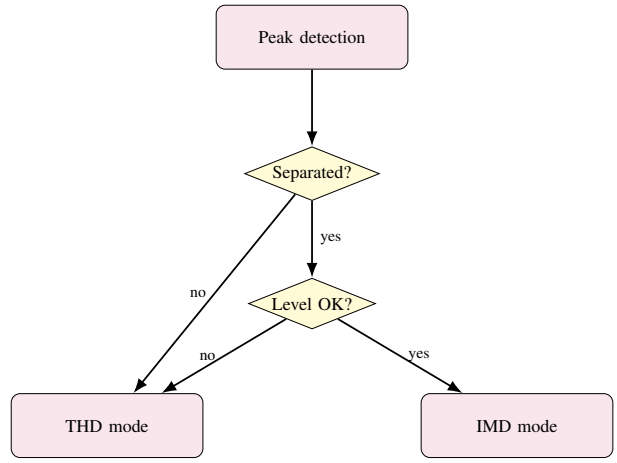

Fig. 3. Automatic selection between THD and IMD modes.

B. Time-Domain Metrics

From the raw measurement block, the software computes peak-to-peak voltage, RMS voltage, peak power, and RMS power across a user-defined load resistance R_L:

$$V_{\mathrm{PP}} = \max(x) - \min(x) \tag{4}$$

$$V_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_n x[n]^2} \tag{5}$$

$$P_{\mathrm{RMS}} = \frac{V_{\mathrm{RMS}}^2}{R_L}. \tag{6}$$

C. Distortion and Noise Metrics

After spectral masks partition the signal into fundamental, distortion, and noise terms with powers V_{fund}, V_{dist}, and V_{noise}, the primary metrics are:

$$\mathrm{THD} = \sqrt{\frac{V_{\mathrm{dist}}}{V_{\mathrm{fund}}}} \tag{7}$$

$$\mathrm{THD+N} = \sqrt{\frac{V_{\mathrm{dist}} + V_{\mathrm{noise}}}{V_{\mathrm{fund}}}} \tag{8}$$

$$\mathrm{SNR} = 20\log_{10}\left(\sqrt{\frac{V_{\mathrm{fund}}}{V_{\mathrm{noise}}}}\right) \tag{9}$$

$$\mathrm{SINAD} = 20\log_{10}\left(\sqrt{\frac{V_{\mathrm{fund}}}{V_{\mathrm{dist}} + V_{\mathrm{noise}}}}\right) \tag{10}$$

$$\mathrm{ENOB} = \frac{\mathrm{SINAD} - 1.76}{6.02}. \tag{11}$$

In two-tone mode, the same computational structure is used, but the distortion mask targets selected intermodulation products rather than harmonic bins.

V. Tone Detection and Operating Modes

YAAR automatically switches between single-tone and two-tone operation. It searches for the two strongest frequency bins inside the active analysis band, enforces a minimum frequency separation, and requires the second tone to remain within a user-configurable relative level threshold.

A. Single-Tone Mode

In single-tone mode, the dominant peak is treated as the carrier. A shifted reference kernel estimates the spectral footprint of the windowed sinusoid, and harmonics up to the limit set by `-thd` are included in the distortion mask.

B. Two-Tone Mode

In two-tone mode, YAAR builds masks for both fundamentals and a selected set of intermodulation products based on combinations of the two detected tones. The implemented products correspond to common low-order IMD terms derived from f_1 and f_2.

VI. User Interface

The display contains three logical regions:

1) a time-domain plot showing the waveform within the selected time span,
2) a logarithmic frequency-domain plot normalized to the current spectral maximum, and
3) a status block containing frequency, distortion, noise, ENOB, voltage, power, and reference frequency indicators.

In single-tone mode, spectral annotations label harmonic numbers. In two-tone mode, the principal tones are labeled `F1` and `F2`. The display updates continuously until the measurement duration expires, the plot window is closed, or the user interrupts execution.

VII. Command-Line Interface

Table I summarizes the public command-line arguments.

VIII. Practical Usage

A. Listing Devices

Before using live capture, identify the desired audio input device:

```
python yaar.py --list
```

B. Basic Live Measurement

A typical real-input run at 192 kHz with an 8 Ω load is:

```
python yaar.py --freq 192000 --chunk 65536 --rload 8
```

C. Single-Tone Simulation

To validate the software path without audio hardware:

```
python yaar.py --simfreq 1000
```

D. Noisy and Distorted Simulation

```
python yaar.py --simfreq 1000 --simnoise -90 --
    simhmncs 0.01
```

E. Two-Tone IMD Measurement

```
python yaar.py --simfreq 1000 --simfreq2 1200 --
    simamp2 1.0
```

F. CSV Logging and Plot Export

```
python yaar.py --simfreq 1000 --csv results.csv --
    plot spectrum.png
```

When the detected carrier remains stable for the required number of iterations, YAAR appends a CSV record and optionally saves plot images.

If plot export is enabled (via `-plot`), the analyzer generates one output image per detected frequency configuration. The file names follow the pattern:

`<base>_<freq1>hz_<freq2>hz.png`

where `<base>` is the user-provided path and `freq1`, `freq2` are the detected fundamental frequencies. In single-tone mode, the second frequency is set to zero.

This behavior allows straightforward automated analysis and batch processing, as multiple measurement conditions produce uniquely identifiable output files.

IX. CSV Output Format

When a CSV path is provided and the file does not yet exist, YAAR writes a header row with the following fields:

```
Mode, F1_hz, F2_hz, THDN_dB, THDN_pct, THD_dB,
        THD_pct,
IMDN_dB, IMDN_pct, IMD_dB, IMD_pct, SINAD_db,
        SNR_dB, ENOB,
    Vrms, Prms, FFT size, Sample Rate
```

The `Mode` field is set to `THD` or `IMD`. In single-tone operation, the second-frequency field is zero.

X. Simulation Model

The simulator can synthesize the following scenarios:

- a single sinusoid with random phase,
- an optional second sinusoid with independent random phase,
- additive white Gaussian noise, and
- either harmonic distortion or selected IMD products scaled to a target relative amplitude.

When only one simulated tone is present, the distortion generator creates harmonic terms from the second through the user-selected harmonic count. When two tones are present, the same parameter instead scales a predefined set of low-order intermodulation products.

XI. Window Functions

YAAR supports Bartlett, Blackman, Hamming, Hann/Hanning, and rectangular windows. The selected window influences leakage, amplitude accuracy, and noise bandwidth. The default Hann window is a balanced choice for general-purpose measurement because it provides good sidelobe suppression while preserving straightforward interpretation.

TABLE I
YAAR COMMAND-LINE OPTIONS.

Option	Meaning	Default
-list	List available sound devices and exit.	false
-freq	Sampling rate in hertz.	192000
-dev	Audio device index for live capture.	4
-chsel	Zero-based selected channel.	1
-chnum	Number of audio channels to open.	2
-chunk	Requested FFT size; rounded to nearest power of two internally.	65536
-adcrng	ADC full-scale voltage range.	100.0
-vrange	Display voltage range for the time plot, shown as $\pm$ value.	40.0
-frange	Upper frequency shown in the spectrum plot. Lower bound is fixed at 10 Hz.	70000.0
-trange	Time span displayed in milliseconds.	100.0
-wrange	Lower dB bound of the spectral plot. Upper bound is 0 dB.	-150.0
-rload	Load resistance used for power calculations.	8.0
-thd	Number of harmonic orders included in THD mode.	7
-duration	Maximum runtime in seconds.	240
-plot	Base path for exported plot image. Saved after signal stabilization.	empty
-csv	CSV path for appending measurement records.	empty
-window	Window function: bartlett, blackman, hamming, hanning/hann, rect.	hanning
-simfreq	Fundamental frequency for simulation mode. If positive, live input is bypassed.	0.0
-simnoise	Simulated white-noise amplitude in dB relative scale.	-160.0
-flttsh	Threshold used to convert reference kernels into binary masks, in dB.	120.0
-cftsh	Center-frequency threshold parameter stored in configuration.	0.25
-twotone-rel-db	Maximum allowed level drop for the second tone to enter IMD mode.	16.0
-peak-sep-hz	Minimum spacing between detected fundamentals for IMD mode.	20.0
-simfreq2	Second simulated tone frequency.	0.0
-simamp2	Relative amplitude of the second simulated tone.	1.0
-simhmncs	Simulated distortion strength for harmonics or IMD products.	0.0

XII. Recommendations for Accurate Measurements

- Use a sufficiently large FFT size, such as 65536 or larger when latency permits.
- Keep the input signal below clipping limits in both the analog front end and the digital capture path.
- For best frequency estimation and leakage control, choose stable sources and appropriate windowing.
- Verify channel selection and device index before assuming a measurement issue.
- In two-tone testing, ensure the tones are separated by more than `-peak-sep-hz` and that the second tone remains within the configured relative level threshold.

XIII. Limitations and Implementation Notes

The following implementation details are important to understand:

- The requested FFT size is rounded to the nearest power of two.
- The displayed frequency range always starts at 10 Hz, regardless of the upper bound selected with `-frange`.
- Spectrum values are displayed relative to the current maximum in the visible band rather than as absolute calibrated dBV or dBFS.
- CSV writing and plot saving occur only after a stability counter reaches its threshold.
- The audio interface itself is delegated to the external `audio_backend` module.

A small code-level observation is worth noting for users interpreting exported CSV data: the CSV header places `SINAD_db` before `SNR_dB`, while the write function currently writes the values in the order `snr_db` followed by `sinad_db`. If strict column alignment matters for downstream analysis, verify this mapping in your post-processing pipeline.

XIV. Signal Generator Utility

A. Overview

In addition to the YAAR analyzer, the project provides a separate command-line program (`gen.py`) for real-time audio signal generation. The generator is an independent tool and can be used either together with YAAR or as a standalone excitation source for external measurement setups.

Its primary purpose is to provide controlled and repeatable test signals for:

- distortion and noise measurements,
- intermodulation testing,
- system validation of ADC/DAC chains, and
- calibration and verification tasks.

B. Relationship to YAAR

The generator and analyzer are intentionally decoupled:

- **YAAR (`yaar.py`)** performs acquisition, FFT analysis, and metric extraction.
- **Generator (`gen.py`)** produces deterministic excitation signals.

They may be used:

- independently, or
- together as a measurement pair (e.g., generator output looped into analyzer input).

TABLE II
SIGNAL GENERATOR COMMAND-LINE OPTIONS.

Option	Description
`-list`	List available output devices
`-dev`	Output device index
`-rate`	Sampling rate (Hz)
`-chunk`	Block size (samples)
`-freq`	Primary tone frequency (Hz)
`-freq2`	Secondary tone frequency (Hz)
`-ratio`	Amplitude ratio A/B
`-level`	Output amplitude scaling (0..1)
`-time`	Run time in seconds (0 = continuous)

YAAR also includes an internal simulation mode; however, the external generator enables testing of full hardware signal paths.

C. Design Principles

The generator uses a *phase accumulator* architecture to ensure:

- frequency accuracy independent of buffer size,
- phase continuity across blocks,
- absence of discontinuities or clicks, and
- deterministic long-duration behavior.

D. Signal Model

The generated waveform consists of one or two sinusoidal components:

$$x[n] = A_1 \sin(\phi_1[n]) + A_2 \sin(\phi_2[n]) \tag{12}$$

with phase evolution:

$$\phi_k[n+1] = \phi_k[n] + \frac{2\pi f_k}{f_s}, \quad k \in \{1, 2\}. \tag{13}$$

The second tone is optional and defined via an amplitude ratio:

$$A_2 = \frac{A_1}{r} \tag{14}$$

The output is scaled by a global level parameter:

$$y[n] = \alpha \cdot x[n], \quad \alpha \in [0, 1]. \tag{15}$$

E. Implementation

The generator is implemented as a `SineGenerator` class maintaining:

- phase accumulators,
- phase increments derived from frequency and sample rate,
- amplitude scaling for multi-tone operation.

Audio output is produced as a continuous stream of 32-bit floating-point samples.

F. Command-Line Interface

The `-time` parameter enables bounded-duration execution, which is particularly useful for automated testing and scripting. When specified, the generator runs for the given number of seconds and then exits cleanly, allowing integration into batch pipelines or regression test frameworks. A value of `0` results in continuous operation until interrupted.

G. Example Usage

```
# List output devices
python gen.py --list

# Single-tone signal
python gen.py --dev 2 --freq 1000

# Two-tone IMD test
python gen.py --dev 2 --freq 1000 --freq2 1200 --
    ratio 1

# Limited duration test
python gen.py --dev 2 --freq 1000 --time 10
```

H. Limitations

- Single-channel output
- No built-in dithering or noise shaping
- Performance limited by Python execution at very high rates

I. Summary

The generator complements YAAR by providing a stable and controllable excitation source. While YAAR supports internal simulation, the external generator enables full end-to-end testing of real-world signal chains.

XV. REQUIREMENTS AND EXECUTION

A. Software Requirements

Both YAAR (`yaar.py`) and the signal generator (`gen.py`) are implemented in Python and require:

- Python 3.9 or newer
- `numpy` for numerical processing
- `matplotlib` for visualization (YAAR)
- an audio I/O library (e.g. `pyaudio` or equivalent backend)

B. Hardware Requirements

- For analysis: a compatible audio input device
- For signal generation: a compatible audio output device
- For loopback testing: physical or virtual connection between output and input

C. Program Execution

Both tools are executed independently from the command line:

```
# Run analyzer
python yaar.py [options]

# Run generator
python gen.py [options]
```

D. Combined Operation

A typical measurement setup uses both tools together:

- `gen.py` produces the excitation signal
- the signal is routed (physically or virtually) to the input
- `yaar.py` performs spectral analysis and metric extraction

XVI. Example Measurement Result

A. Single tone THD example

Figure 4 shows a representative measurement obtained using the internal simulation mode with a non-integer tone frequency, added harmonic distortion, and elevated noise floor.

```
python yaar.py --simfreq 987.3 --simhmncs 0.01 --
    simnoise -60
```

The resulting spectrum illustrates the detected fundamental, harmonic components, and noise distribution as processed by the FFT-based analysis pipeline.

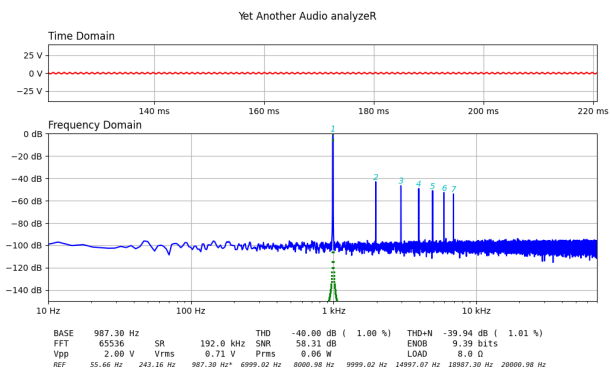

Fig. 4. Example spectrum for simulated input at 987.3 Hz with added distortion and noise.

No direct coupling between the programs is required.

B. Two-Tone IMD Example

Figure 5 shows a simulated two-tone measurement illustrating intermodulation distortion behavior. Two tones at low and high frequency are combined with added distortion and noise.

```
python yaar.py --simfreq 55.66 --simfreq2 6999.02 --
    simhmncs 0.01 --simnoise -40
```

The resulting spectrum demonstrates automatic transition into IMD mode, with identification of the two fundamental tones and the appearance of intermodulation products generated by the nonlinear distortion model.

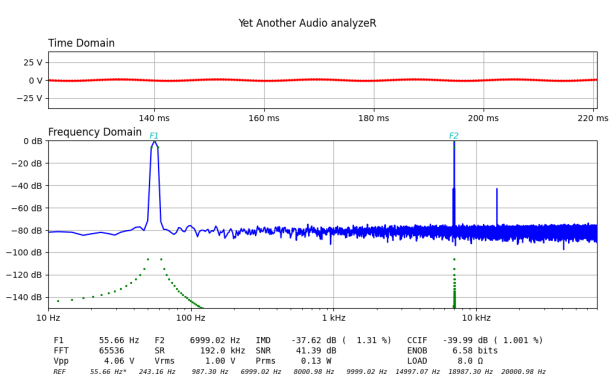

Fig. 5. Two-tone IMD example with tones at 55.66 Hz and 6999.02 Hz, including distortion and noise.

C. Hardware Single-Tone Measurement Example

Figure 6 shows a measurement performed using a UNI-T UTG932E signal generator, configured at 987.3 Hz, and connected to a COSMOS E1DA ADC.

This setup represents a real hardware measurement chain, where the generator provides a stable excitation signal and the ADC captures the response for FFT-based analysis in YAAR. The resulting spectrum demonstrates practical system performance, including noise floor and distortion components introduced by the physical signal path.

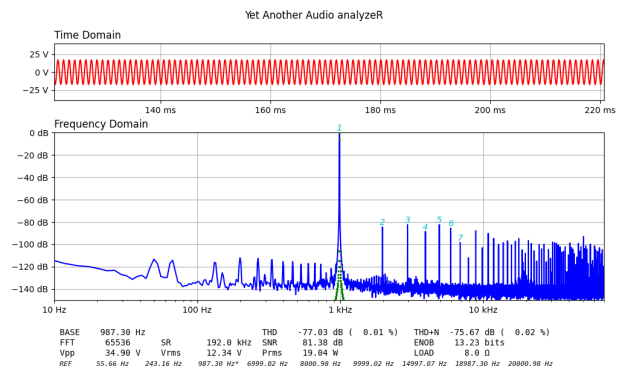

Fig. 6. Hardware measurement using a UNI-T UTG932E signal generator at 987.3 Hz and a COSMOS E1DA ADC.

D. Hardware Two-Tone IMD (CCIF) Measurement Example

Figure 7 shows a two-tone measurement using a FiiO BR13 connected to a COSMOS E1DA ADC. The excitation signal was generated using:

```
python gen.py --dev 0 --freq 243.16 --freq2 6999.02
```

This configuration produces a CCIF-style intermodulation test, where two widely spaced tones are used to evaluate nonlinear behavior. The resulting spectrum allows observation of intermodulation distortion (IMD) products, including sum and difference components, as well as higher-order mixing artifacts.

Both IMD and CCIF-related distortion components can be identified and evaluated directly from the spectral output.

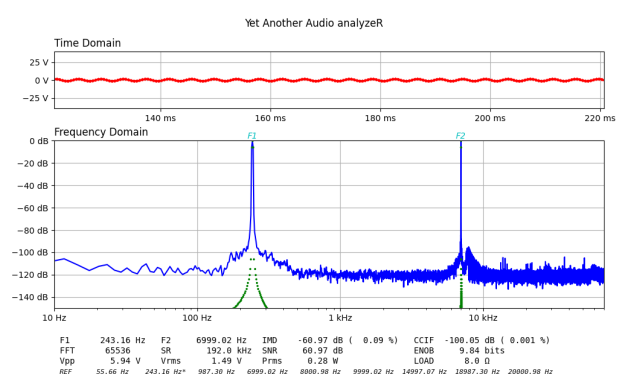

Fig. 7. Two-tone IMD (CCIF) measurement using FiiO BR13 and COSMOS E1DA ADC with tones at 243.16 Hz and 6999.02 Hz.

XVII. Troubleshooting

A. No Input or Empty Display

Check the selected device index with `-list`, verify the correct channel number, and confirm that the chosen device supports the requested sample rate.

B. Unexpected Noise Floor

Increase the FFT size, confirm proper grounding and shielding in the analog setup, and inspect whether the selected window is appropriate for the stimulus.

C. Two-Tone Mode Does Not Trigger

Reduce `-twotone-rel-db` only if the second tone is intentionally weak, and ensure that the tone separation exceeds `-peak-sep-hz`.

D. Power Values Seem Incorrect

Verify the configured load resistance `-rload` and confirm that the input voltage scaling in the backend is calibrated correctly.

XVIII. Conclusion

YAAR is a compact but capable audio measurement tool that integrates real-time visualization with practically relevant spectral metrics. Its combination of a live backend, a synthetic signal path, automatic operating-mode selection, rolling FFT averaging, and export support makes it useful for rapid evaluation of amplifiers, ADCs, DACs, and other audio signal chains. The architecture is intentionally simple and transparent, which also makes the program a suitable starting point for custom extensions.

Appendix: Quick Command Reference

```
# List audio devices
python yaar.py --list

# Live measurement
python yaar.py --freq 192000 --dev 4 --chsel 1 --
    chnum 2

# Single-tone simulation
python yaar.py --simfreq 1000

# THD-style simulation with noise and harmonics
python yaar.py --simfreq 1000 --simnoise -100 --
    simhmncs 0.02

# Two-tone IMD simulation
python yaar.py --simfreq 19000 --simfreq2 20000 --
    simamp2 1.0

# Export CSV and plot
python yaar.py --simfreq 1000 --csv out.csv --plot
    out.png
```